2.6. Utilización de bucles (Criterio 2.f)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.f** (*Se han utilizado bucles y se ha verificado su funcionamiento*).

- **Ponderación del Criterio:** (**20% del RA2** | **1% sobre la calificación final del módulo**).

A. ¿Qué es un Bucle y por qué se necesita?
En programación, un **bucle** o **estructura repetitiva** es un mecanismo que permite ejecutar un mismo bloque de sentencias varias veces de forma automática mientras se cumpla una condición determinada.

Sin bucles, si quisiéramos procesar una lista de 50 facturas, tendríamos que repetir el mismo bloque de código 50 veces en el archivo, lo que generaría un código imposible de mantener. Los bucles resuelven este problema parametrizando la repetición.

En el desarrollo cliente, los bucles resultan imprescindibles para recorrer colecciones de datos, generar dinámicamente nodos en el DOM de la página web y procesar respuestas estructuradas procedentes de servicios externos.

B. El Bucle while (Comprobación al Inicio)
El bucle while comprueba su condición **antes** de ejecutar cualquier instrucción del bloque. Si la condición inicial se evalúa a false, el cuerpo del bucle **no se ejecutará ninguna vez (0 iteraciones)**.

Sintaxis canónica:

```
while (condicion) {
  // Instrucciones que se repiten mientras la condición sea true
  // Instrucción de avance/actualización (OBLIGATORIA)
}
```

Ejemplo guiado:

```
let pase = 0; // 1. Inicialización de la variable de control

// 2. Condición de entrada: mientras 'pase' sea menor que 5
while (pase < 5) {
  console.log(`Pase número ${pase + 1}`); // 3. Cuerpo del bucle

  pase++; // 4. Modificación de la variable de control (+1)
}

console.log("Fin del bucle while.");
```

El peligro del Bucle Infinito:

Si se olvida la línea pase++;, la variable siempre valdrá 0. La condición 0 < 5 será siempre verdadera (true), y el bucle jamás alcanzará su condición de salida, congelando por completo la pestaña del navegador.

C. El Bucle do...while (Comprobación al Final)

El bucle do...while es una variante donde la condición se evalúa **después** de ejecutar el bloque de instrucciones. Por este motivo, garantiza que el cuerpo del bucle **se ejecuta siempre al menos una vez (**1 **o más iteraciones)**, incluso si la condición resulta ser falsa desde el primer instante.

Sintaxis canónica:

```
do {
  // Instrucciones (se ejecutan al menos una primera vez)
} while (condicion);
```

Ejemplo guiado: Búsqueda de los primeros 5 números pares

```
let contadorPares = 0; // Controla cuántos pares hemos encontrado
let numeroActual = 1;  // Número que vamos examinando

do {
  // Comprobamos si el número actual es par con el operador módulo
(%)
  if (numeroActual % 2 === 0) {
    console.log(`Número par detectado: ${numeroActual}`);
    contadorPares++; // Incrementamos el contador de pares
encontrados
  }

  numeroActual++; // Avanzamos al siguiente número entero
} while (contadorPares < 5); // Condición evaluada al final

console.log("Se han encontrado los 5 primeros números pares.");
```

Comparativa: while vs. do...while

```
let llaveAcceso = false;

// 1. Con bucle while (Pre-test):
while (llaveAcceso === true) {
  console.log("Esto NUNCA se imprimirá (0 vueltas)");
}

// 2. Con bucle do...while (Post-test):
do {
  console.log("Esto se imprime EXACTAMENTE 1 VEZ a pesar de ser false");
```

} while (llaveAcceso === true);

D. El Bucle for (Determinado por Contador)

El bucle for se emplea cuando se conoce de antemano el número de iteraciones o cuando se trabaja con contadores numéricos estructurados. Compacta en una sola línea la inicialización, la condición y la actualización.

Estructura tripartita:

```
for (expresionInicial; expresionCondicional; expresionDeActualizacion) {
  // Bloque de instrucciones a repetir
}
```

1. **expresionInicial:** Se ejecuta una sola vez al arrancar el bucle (generalmente declara la variable contadora usando let para aislar su ámbito garantizando que desaparezca de la memoria una vez finalizado el bucle).
2. **expresionCondicional:** Se evalúa antes de cada vuelta; si devuelve true, entra al cuerpo; si devuelve false, termina el bucle.
3. **expresionDeActualizacion:** Se ejecuta automáticamente al terminar el cuerpo en cada iteración (generalmente un incremento i++ o decremento i--).

Ejemplo guiado: Generador de tablas de multiplicar

```
const FACTOR = 9;

// for (inicialización; condición; actualización)
for (let i = 1; i <= 10; i++) {
  let resultado = FACTOR * i;
  console.log(`${FACTOR} x ${i} = ${resultado}`);
}

// bucle for decreciente
for (let i = 10; i >= 0; i--) {
  console.log(i); // Cuenta del 10 al 0
}

// Recorrer un Array al revés
const herramientas = ["Martillo", "Destornillador", "Alicates"];
// Iniciamos en el último índice y bajamos hasta el primero (0)
for (let i = herramientas.length - 1; i >= 0; i--) {
  console.log(`Índice ${i}: ${herramientas[i]}`);
}

// Ir de 5 en 5
for (let i = 20; i >= 0; i -= 5) {
  console.log(i); // 20, 15, 10, 5, 0
}
```

Omitir la inicialización: Si la variable ya ha sido inicializada antes del bucle:

```
coches = ["Toyota", "Kia", "Citroen", "Ford"]
var i = 2;
var len = coches.length;
for (; i < len; i++) {
  console.log(coches[i]);
}
```

Omitir la condición: Si se omite, el motor asume `true` indefinidamente, exigiendo un `break` interno para no generar un bucle infinito.

```
for (var k = 0; ; k++) {
  if (k > 10) break; // Salida controlada
  console.log(k);
}
```

Omitir el incremento: Se puede desplazar el incremento al interior del cuerpo del bucle:

```
j = 0;
for (; j < 5; ) {
  console.log("Valor:", j);
  j++; // Incremento gestionado manualmente dentro del bloque
}
```

E. Bucles Modernos de Colección: for...of y for...in

1. for...of (Recorrido de Valores en Iterables)

Introducido en ES6, es la estructura recomendada para recorrer los **valores** de arrays o caracteres de una cadena sin necesidad de gestionar contadores manuales:

```
const lenguajes = ["JavaScript", "TypeScript", "Python"];

for (const elemento of lenguajes) {
  console.log("Lenguaje en catálogo:", elemento);
}
// Salida: JavaScript, TypeScript, Python
```

2. for...in (Recorrido de Claves en Objetos)

Recorre las **propiedades o claves** de un objeto literal:

```
const configuracion = { servidor: "localhost", puerto: 8080, ssl:
true };

for (const propiedad in configuracion) {
  console.log(`Clave: ${propiedad} | Valor:
${configuracion[propiedad]}`);
}
```

No debe usarse para recorrer arrays, ya que itera sobre los índices como cadenas de texto (`"0"`, `"1"`, etc.) y puede recuperar propiedades adicionales o prototipos añadidos accidentalmente al objeto `Array`.

Los 3 problemas de `for...in` en Arrays

- **Tipado incorrecto:** Los índices se devuelven como cadenas de texto (`"0"`, `"1"`) en lugar de números. Si haces una operación matemática (como `index + 1`), terminarás con una concatenación de texto (`"01"`).
- **Contaminación del prototipo:** Si una librería externa o un script añade un método al prototipo global (`Array.prototype.miMetodo = ...`), `for...in` lo listará como si fuera un elemento más del array.
- **Falta de orden garantizado:** Técnicamente, no garantiza mantener el orden cronológico o numérico de los elementos.

```
// Añadimos una propiedad al prototipo


Array.prototype.nuevoMetodo = "¡Soy un nuevo método!";

const colores = ["rojo", "azul"];

for (const indice in colores) {
  console.log(`Índice: ${indice} (Tipo: ${typeof indice}) ->
Valor: ${colores[indice]}`);

}
```

3. forEach (El método `.forEach()` es la alternativa funcional estándar en JavaScript para recorrer un array cuando necesitas realizar una acción por cada elemento, ofreciendo acceso nativo tanto al valor como al índice correcto.

A diferencia de `for...in`, `.forEach()` **garantiza que el índice sea un número**, respeta el orden del array y ignora por completo las propiedades añadidas al prototipo.

```
Array.prototype.nuevoMetodo = "¡Soy un nuevo método!";

const colores = ["rojo", "azul", "verde"];

colores.forEach((color, index) => {

  console.log(`Índice: ${index} (Tipo: ${typeof index}) -> Valor:
${color}`);

});
```

Aunque es seguro frente a la contaminación del prototipo, debes tener en cuenta tres reglas estrictas antes de usarlo:

1. **No puedes usar `break` ni `continue`:** Si necesitas detener el bucle a la mitad o saltarte una iteración basándote en una condición, `.forEach()` no lo permite. Tienes que usar `for...of` para ello.
2. **Ignora los elementos vacíos (Sparse Arrays):** Si un array tiene "huecos" (por ejemplo, `[1, , 3]`), `.forEach()` se saltará la posición vacía, mientras que un bucle clásico o un `for...of` devolverá `undefined`.

F. Estructuras de Salto y Control del Bucle

Las sentencias de salto alteran el flujo iterativo normal ante situaciones imprevistas o criterios de corte.

1. break (Interrupción Inmediata)

Aborta el bucle por completo y transfiere el control a la línea de código situada inmediatamente después del cierre de llaves:

```
for (let i = 1; i <= 10; i++) {
  if (i === 6) {
    console.log("Corte de emergencia: bucle interrumpido en i =
6");
    break; // Sale definitivamente del for
  }
  console.log(`Vuelta actual: ${i}`);
}
// Solo imprime del 1 al 5
```

2. continue (Salto a la Siguiente Vuelta)

Descarta las sentencias que queden por ejecutar en la iteración actual y salta de inmediato a la siguiente evaluación condicional (y actualización en for):

```
// Imprimir solo números impares descartando los pares con
continue:
for (let n = 1; n <= 6; n++) {
  if (n % 2 === 0) {
    continue; // Descarta el resto del bloque para este número par
  }
  console.log(`Impar: ${n}`); // Imprime 1, 3, 5
}
```

3. Declaraciones Etiquetadas (*Labeled Statements*)

Asocian un identificador a un bucle externo para permitir que un break o continue situado en un bucle anidado rompa el ciclo principal exterior sin usar variables bandera intermedias:

```
let objetivoEncontrado = false;

bucleMatriz: for (let fila = 1; fila <= 3; fila++) {
  for (let columna = 1; columna <= 3; columna++) {
    console.log(`Inspeccionando celda [${fila}, ${columna}]`);

    if (fila === 2 && columna === 2) {
      console.log("Coincidencia detectada. Abortando matriz al
completo.");
      break bucleMatriz; // Rompe el for exterior etiquetado
    }
  }
}
```

`break` no solo interrumpe bucles y `switch`, sino que en JavaScript existe la sintaxis de **etiquetas (*labels*)** para romper un bloque de código arbitrario o escapar de bucles fuertemente anidados:

```
//bloque etiquetado:
miEtiqueta: {
  console.log("Inicio");
  break miEtiqueta; // Salta fuera del bloque etiquetado
  console.log("Esto nunca se ejecutará");
}

var coches = ["BMW", "Volvo", "Saab", "Ford"];
var texto = "";

list: {
  texto += coches[0] + " ";
  texto += coches[1] + " ";
  break list; // Rompe la ejecución del bloque 'list' aquí
  texto += coches[2] + " ";
  texto += coches[3] + " ";
}

console.log(texto); // Muestra únicamente: "BMW Volvo "
```

G. Actividades Prácticas

Actividad 1: "El Bug Silencioso"

Lee detenidamente el siguiente bloque de código. Tu objetivo es realizar una **traza de ejecución mental** (sin usar la consola del navegador) para predecir el resultado de salida. Explica que ocurre. Har lo mismo con un bucle decreciente.

```
// Estado inicial del array
const numeros = [2,4,6,8,9];

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    numeros.splice(i, 1);
  }
}

console.log(numeros); // Resultado: [4, 6, 9]
```

Sintaxis: `array.splice(inicio, cuántosBorrar, elemento1, elemento2...)`

Ejemplo A: Solo borrar elementos

```
const letras = ["A", "B", "C", "D"];

// Desde el índice 1, borra 2 elementos
letras.splice(1, 2);

console.log(letras); // ["A", "D"] (El array original HA CAMBIADO)
```

Ejemplo B: Insertar elementos sin borrar nada

```
const colores = ["Rojo", "Azul"];

// En el índice 1, borra 0 elementos e inserta "Verde"
colores.splice(1, 0, "Verde");

console.log(colores); // ["Rojo", "Verde", "Azul"]
```

Ejemplo C: Reemplazar elementos

```
const numeros =;

// En el índice 1, borra 1 elemento (el 20) e inserta el 99
numeros.splice(1, 1, 99);
```

```
console.log(numeros); // [10, 99, 30]
```

Actividad 2: "El Algoritmo del Rombo"

Construye un script que dibuje en la consola una figura simétrica de rombo mediante bucles `for` anidados, controlando los espacios y caracteres de asterisco (`*`)

Actividad 3:"Validación de Entrada con `do...while` "

Implementa un mecanismo que solicite un código numérico de 4 dígitos mediante `prompt()`, repitiendo la petición con `do...while` mientras el dato introducido no sea numérico o tenga una longitud distinta a 4 caracteres.